

ARENACUP OS

Documento de Especificação Técnica e Escopo Completo do Projeto

1. VISÃO GERAL DO ECOSISTEMA

O ArenaCup OS é um ecossistema distribuído de alta escalabilidade orientado a eventos e microsserviços especializado, projetado de forma nativa para sustentar as operações críticas de volumetria de acessos e volatilidade transacional da Copa do Mundo da FIFA. O sistema adota padrões arquiteturais corporativos baseados na plataforma Java para mitigar pontos de falha única, assegurar consistência eventual e promover escalabilidade linear sob cargas extremas.

2. INFRAESTRUTURA E PILHA TECNOLÓGICA GLOBAL

A malha de infraestrutura global do projeto é padronizada para garantir a governança, containerização e resiliência de todos os componentes distribuídos. Toda a solução é estruturada sob a seguinte especificação técnica unificada:

- **Linguagem Base:** Java (Spring Boot)
- **Persistência Relacional:** Spring Data JPA
- **Persistência Não-Relacional:** Spring Data MongoDB
- **Mecanismo de Cache e Travas:** Redis
- **Espinha Dorsal de Mensageria:** Apache Kafka
- **Orquestração de Contêineres:** Kubernetes
- **Segurança e Roteamento Periférico:** API Gateway
- **Distribuição de Tráfego:** Load Balancer
- **Padrão de Comunicação Síncrona:** API REST via Spring RestClient

3. ESPECIFICAÇÃO DE MICROSERVIÇOS E INDEXAÇÃO DE STACKS

O ecossistema é segmentado em sete unidades funcionais independentes e isoladas em nível de domínio de dados e infraestrutura computacional:

3.1. ms-core-data (Núcleo de Dados Centralizado)

Responsável pelo armazenamento centralizado e fornecimento estável de dados mestres imutáveis ou de baixa mutabilidade estrutural compartilhados por todo o sistema (cadastros de federações, seleções oficiais, dados técnicos de estádios e delegações).

Stacks associadas: JAVA SPRINGBOOT Spring data jpa Kubernetes API REST

3.2. ms-tickets (Módulo de Vendas e Reserva de Ingressos)

Unidade transacional crítica com a maior pressão de concorrência simultânea. Gerencia o inventário global de assentos e processamento de ordens de pagamento, implementando mecanismos de concorrência estrita para mitigar a ocorrência de venda duplicada (double-booking).

Stacks associadas: JAVA SPRINGBOOT Spring data jpa redis kafka kubernetes api gateway load balancer API REST

3.3. ms-matches (Módulo de Gestão de Partidas e Placar Oficial)

Responsável pelo controle do estado ativo dos jogos, escalões e estatísticas ao vivo. Executa chamadas à API externa via RestClient do Spring e propaga alterações transacionais por meio de eventos no Kafka e streaming de dados.

Stacks associadas: JAVA SPRINGBOOT Spring data MongoDB redis kafka kubernetes API REST

3.4. ms-access-control (Controle de Catracas e Acesso aos Estádios)

Unidade focada no processamento periférico de validação física de ingressos via QR Code nos estádios. Exige respostas em milissegundos e isolamento de falhas para operação local em caso de degradação da conectividade central.

Stacks associadas: JAVA SPRINGBOOT Spring data jpa kafka kubernetes load balancer API REST

3.5. ms-engagement (Gamification e Votação de "Craque do Jogo")

Garante o processamento massivo e veloz de votações de engajamento do público concentradas nos períodos finais das partidas. Prioriza a velocidade de gravação e agregação instantânea.

Stacks associadas: JAVA SPRINGBOOT redis kafka kubernetes API REST

3.6. ms-logistics (Gestão de Delegações, Treinos e Transporte Oficial)

Orquestra o agendamento de recursos complexos, campos de treinamento, hotéis e frotas de transporte de forma a evitar conflitos operacionais entre as 32 delegações.

Stacks associadas: JAVA SPRINGBOOT Spring data jpa kafka kubernetes API REST

3.7. ms-analytics (Consolidador Financeiro e Taxa de Ocupação)

Componente estritamente reativo alimentado por fluxos contínuos de dados assíncronos. Consolida métricas financeiras globais e monitora a ocupação de estádios em tempo real dividida por setores.

Stacks associadas: JAVA SPRINGBOOT Spring data jpa redis kafka kubernetes API REST

4. MODELAÇÃO E ESTRUTURA DE DADOS (PERSISTÊNCIA POLIGLOTA)

A arquitetura adota Persistência Poliglota para mitigar gargalos estruturais e adequar o modelo de dados às necessidades transacionais e analíticas de cada contexto delimitado.

4.1. Abordagem Relacional (PostgreSQL via Spring Data JPA)

Utilizada onde há exigência de conformidade ACID estrita e integridade referencial forte. O modelo centralizado do ms-core-data e ms-tickets baseia-se em tabelas estruturadas com chaves estrangeiras rígidas.

4.2. Abordagem Não-Relacional (MongoDB via Spring Data MongoDB)

O ms-matches adota o modelo NoSQL baseado em documentos para o gerenciamento dinâmico da linha do tempo das partidas (Match Timeline). A API externa de dados ao vivo (worldcup2026) fornece estruturas fluidas que são mapeadas diretamente em documentos estruturados sem a necessidade de operações custosas de JOIN.

Exemplo conceitual do documento de persistência da partida na coleção matches:

```
{
  "_id": "match_2026_01",
  "matchday": "1",
  "group": "A",
  "stadium_id": "101",
  "home_team": { "team_id": "ARG", "name_en": "Argentina", "score": 2 },
  "away_team": { "team_id": "FRA", "name_en": "France", "score": 1 },
  "timeline_events": [
    { "type": "GOAL", "minute": 12, "player": "L. Messi", "team_id": "ARG" },
    { "type": "YELLOW_CARD", "minute": 43, "player": "A. Rabiot", "team_id": "FRA" },
    { "type": "GOAL", "minute": 68, "player": "K. Mbappé", "team_id": "FRA" }
  ],
  "finished": false,
  "local_date": "June 11, 2026"
}
```

4.3. Estruturas em Memória (Redis)

O Redis desempenha papel duplo: controle transacional através de travas distribuídas baseadas em semáforos de tempo de expiração no ms-tickets (impedindo dupla reserva) e estruturas do tipo *Sorted Sets* no ms-engagement para computar e ranquear milhões de votos simultâneos em tempo real com complexidade algorítmica $O(\log(N))$.

5. MATRIZ DE RELACIONAMENTO E COMUNICAÇÃO INTER-SERVIÇOS

A comunicação interna é dividida estritamente de acordo com o ciclo de vida e criticidade do fluxo. Operações de consulta usam chamadas síncronas REST via RestClient do Spring, enquanto fluxos de atualização e gatilhos de negócio utilizam mensageria assíncrona orientada a eventos via Apache Kafka.

Microsserviço Origem	Microsserviço Destino	Tipo / Tecnologia	Descrição do Fluxo Operacional
API Gateway	Todos os Serviços	Síncrona / HTTP REST	Ponto de entrada periférico. Realiza rate-limiting, validação de tokens JWT e distribui carga via Load Balancer.

Microsserviço Origem	Microsserviço Destino	Tipo / Tecnologia	Descrição do Fluxo Operacional
ms-tickets	ms-core-data	Síncrona / RestClient	Consulta síncrona para validação estática de dados estruturais de estádios e assentos oficiais antes de iniciar a venda.
ms-tickets	ms-access-control	Assíncrona / Kafka	Publicação do evento TicketIssuedEvent no tópico correspondente. O controle de acesso consome a mensagem para atualizar a base local permitindo o QR Code.
ms-tickets	ms-analytics	Assíncrona / Kafka	Envio imediato de dados financeiros consolidados de venda para a geração de relatórios reativos de receita.
ms-matches	ms-core-data	Síncrona / RestClient	Resolução e consistência dos códigos de seleções e informações base dos estádios nas listagens de partidas.
ms-matches	ms-engagement	Assíncrona / Kafka	Sinaliza o início, intervalos e fim de jogo (MatchStatusChangedEvent) regulando a abertura e fechamento automático das janelas de votação.
ms-access-control	ms-analytics	Assíncrona / Kafka	Gera notificações de eventos de catraca acionada (TurnstileValidatedEvent), permitindo o cálculo de taxa de ocupação em tempo real.
ms-logistics	ms-core-data	Síncrona / RestClient	Verificação cadastral das seleções e locais de treino cadastrados centralizadamente.

6. PLANO DE EXECUÇÃO, NÍVEL DE DIFICULDADE E CRONOGRAMA BACKEND

O desenvolvimento completo da solução backend considerando um grupo de 6 integrantes da Engenharia de Software no prazo fixado de 3 semanas (21 dias) possui um **Nível de Dificuldade Global: ALTO**. O desafio reside na correta orquestração dos ambientes computacionais distribuídos, configuração dos brokers e contenção de concorrência.

6.1. Escalonamento de Complexidade e Carga de Trabalho

- **Alta Densidade de Esforço (ms-tickets & ms-matches):** Demanda majoritária de tempo. O ms-tickets exige calibração fina de travas no Redis e isolamento transacional PostgreSQL. O ms-matches exige a construção da rotina programada (Worker @Scheduled) com o RestClient do Spring para efetuar a ingestão robusta da API externa aberta de maneira resiliente e persistir sem perdas no MongoDB.
- **Média Densidade de Esforço (ms-logistics & ms-access-control):** Exige implementação de padrões distribuídos. O módulo de logística foca no padrão Saga (gerenciamento de estado de transações distribuídas sem acoplamento), enquanto o controle de catracas foca na tolerância a falhas via Circuit Breaker (Resilience4j).

- **Baixa Densidade de Esforço (ms-core-data & ms-analytics):** Lógica predominantemente simples. O núcleo de dados atua como um repositório básico de cadastros relacional e o analytics atua de forma reativa a tópicos, simplificando os testes unitários e de integração.

6.2. Cronograma Detalhado Backend (Prazo Estrito: 3 Semanas)

Semana 1: Infraestrutura Global, Dockerização e Estrutura de Dados Base (Dias 1 - 7)

- Levantamento e provisionamento dos ambientes locais unificados via docker-compose contendo instâncias isoladas de Apache Kafka, Redis, PostgreSQL e MongoDB.
- Inicialização de todos os esqueletos de microserviços Java Spring Boot e parametrização do API Gateway e Load Balancer.
- Desenvolvimento completo e homologação do ms-core-data para disponibilizar endpoints REST base de consulta de estádios e delegações.

Semana 2: Desenvolvimento de Concorrência, Ingestão NoSQL e Mensageria (Dias 8 - 14)

- Implementação lógicas de concorrência no ms-tickets (Redis Distributed Locks e controle transacional JPA).
- Desenvolvimento das classes de comunicação HTTP com o RestClient no ms-matches para conexão à API worldcup2026, tratamento do ciclo de vida de 84 dias do Token JWT e escrita das coleções no MongoDB.
- Criação da topologia e infraestrutura de tópicos no Apache Kafka, padronizando os contratos de eventos serializados em formato JSON.

Semana 3: Padrões de Resiliência Distributed, Transações Complexas e Testes Fim a Fim (Dias 15 - 21)

- Implementação do padrão Saga no ms-logistics para mitigar colisões de horários de treinos através de transações compensatórias acionadas via Kafka.
- Ajustes de tolerância a falhas com isolamento de Circuit Breakers (Resilience4j) no processamento do ms-access-control.
- Execução de testes de integração em lote (fim a fim) simulando o fluxo sequencial: Ingestão de partidas -> Compra de Ingresso -> Atualização de Tópico -> Validação na Catraca -> Consolidação no Analytics.
- Estabelecimento de Code Freeze no Dia 19 para correções finas e estabilização de deploys em ambiente Kubernetes.